Single-Node RAG Deployment Guide - vLLM with MIG

Architecture:

Layer	Component	Details
User Interface	**Open WebUI**	Central hub for chat, RAG orchestration, and model selection.
Inference Layer	**vLLM & Infinity**	Running **gpt-oss** and **Qwen2.5** family and **BAAI** embedding/reranker models.
Orchestration	**Canonical K8s**	**Kubernetes 1.32 (LTS)** with NVIDIA GPU Operator (MIG mixed strategy).
Data Layer	**OpenSearch (Snap)**	Vector Database running on Bare Metal.
Operating System	**Ubuntu 24.04 LTS**	Optimized with NVIDIA Open Drivers and CUDA
Hardware	**Lenovo SR650a V4**	2 x NVIDIA RTX PRO 6000 Blackwell (96GB VRAM each). 1TiB RAM 192 CPU cores

GPU configuration:

Hardware	Allocation	Role	Model
GPU 0	**Full (96GB)**	**Generalist**	**gpt-oss-120B** (FP4)
GPU 1	**MIG Slice A (48GB)**	**Code Specialist**	**Qwen2.5-Coder-32B** (FP8)
GPU 1	**MIG Slice B (24GB)**	**Small Model**	**Qwen2.5-7B-Instruct**
GPU 1	**MIG Slice C (24GB)**	**Embedding Model**	**BAAI/bge-m3 (*)**

(*) not yet supported on RTX 6000 Pro Blackwell), currently running on CPU.

Hardware used for Testing:

1. Lenovo ThinkSystem SR650a V4 powered by 2 x NVIDIA RTX PRO 6000 Blackwell (96GB each)

OS: Ubuntu 24.04 LTS

Performances

This is the result of a non-rigorous performance stress test to have an idea of how many concurrent chats can be served by vLLM with the biggest model implemented (gpt-oss-120B), using an entire physical GPU.

The test has been performed by sending concurrent requests with a prompt taken randomly between 5 possible options.

1. **1 request -> Avg Speed (TPS) 177.48 tokens/sec :**

```
Shell
--- Connecting to vLLM ---
```

```
Targeting Model: gpt-oss-120b
--- Starting 1 randomized stress-tests ---
 -> Request 0 started [Explain the con...]
.......................... [Req 0 Done]


=== RANDOMIZED STRESS TEST RESULTS ===
Avg Latency (TTFT): 0.4406s
Avg Speed (TPS):    177.48 tokens/sec (Per user)
Total Throughput:   167.12 tokens/sec (Aggregate)
Total Tokens:       1261
(benchmark_llm) ubuntu@ai-node:~$ vi benchmark_vllm.py
(benchmark_llm) ubuntu@ai-node:~$ vi benchmark_vllm.py
(benchmark_llm) ubuntu@ai-node:~$ python3 benchmark_vllm.py
```

2. 10 requests -> Avg Speed (TPS) 67.74 tokens/sec:

Shell

```
--- Connecting to vLLM ---
Targeting Model: gpt-oss-120b
--- Starting 10 randomized stress-tests ---
 -> Request 0 started [Write a compreh...]
 -> Request 1 started [Explain the con...]
 -> Request 2 started [Explain the con...]
 -> Request 3 started [Write a sci-fi ...]
 -> Request 4 started [Write a sci-fi ...]
 -> Request 5 started [Write a compreh...]
 -> Request 6 started [Explain the sta...]
 -> Request 7 started [Explain the con...]
 -> Request 8 started [Write a sci-fi ...]
 -> Request 9 started [Write a sci-fi ...]
.....................................................................................
.....................................................................................
.....................................................................
[Req 7 Done]
.......................................... [Req 1 Done]
........ [Req 2 Done]
.................................................... [Req 0 Done]
 [Req 8 Done]
 [Req 3 Done]
 [Req 9 Done]
 [Req 4 Done]
```

```
 [Req 6 Done]
 [Req 5 Done]


=== RANDOMIZED STRESS TEST RESULTS ===
Avg Latency (TTFT): 2.8244s
Avg Speed (TPS):    67.74 tokens/sec (Per user)
Total Throughput:   567.27 tokens/sec (Aggregate)
Total Tokens:       16922
(benchmark_llm) ubuntu@ai-node:~$ vi benchmark_vllm.py
(benchmark_llm) ubuntu@ai-node:~$ python3 benchmark_vllm.py
```

3. 100 requests -> Avg Speed (TPS) 25.92 tokens/sec:

Shell

```
--- Connecting to vLLM ---
Targeting Model: gpt-oss-120b
--- Starting 100 randomized stress-tests ---
 -> Request 0 started [Compare and con...]
 -> Request 1 started [Compare and con...]
 -> Request 2 started [Compare and con...]
 -> Request 3 started [Write a compreh...]
 -> Request 4 started [Write a compreh...]
 -> Request 5 started [Explain the con...]
 -> Request 6 started [Write a compreh...]
 -> Request 7 started [Write a sci-fi ...]
 -> Request 8 started [Explain the con...]
 -> Request 9 started [Explain the sta...]
 -> Request 10 started [Write a sci-fi ...]
 -> Request 11 started [Explain the con...]
 -> Request 12 started [Explain the sta...]
 -> Request 13 started [Compare and con...]
 -> Request 14 started [Explain the sta...]
 -> Request 15 started [Explain the con...]
 -> Request 16 started [Explain the sta...]
 -> Request 17 started [Compare and con...]
 -> Request 18 started [Explain the con...]
 -> Request 19 started [Write a compreh...]
 -> Request 20 started [Explain the sta...]
 -> Request 21 started [Compare and con...]
 -> Request 22 started [Compare and con...]
 -> Request 23 started [Compare and con...]
```

```
-> Request 24 started [Write a compreh...]
-> Request 25 started [Explain the sta...]
-> Request 26 started [Explain the con...]
-> Request 27 started [Write a compreh...]
-> Request 28 started [Explain the sta...]
-> Request 29 started [Compare and con...]
-> Request 30 started [Write a compreh...]
-> Request 31 started [Write a compreh...]
-> Request 32 started [Explain the con...]
-> Request 33 started [Compare and con...]
-> Request 34 started [Explain the sta...]
-> Request 35 started [Write a sci-fi ...]
-> Request 36 started [Explain the con...]
-> Request 37 started [Explain the sta...]
-> Request 38 started [Compare and con...]
-> Request 39 started [Explain the con...]
-> Request 40 started [Write a compreh...]
-> Request 41 started [Write a compreh...]
-> Request 42 started [Compare and con...]
-> Request 43 started [Explain the sta...]
-> Request 44 started [Write a compreh...]
-> Request 45 started [Write a sci-fi ...]
-> Request 46 started [Explain the sta...]
-> Request 47 started [Explain the sta...]
-> Request 48 started [Explain the con...]
-> Request 49 started [Compare and con...]
-> Request 50 started [Compare and con...]
-> Request 51 started [Write a sci-fi ...]
-> Request 52 started [Explain the sta...]
-> Request 53 started [Explain the sta...]
-> Request 54 started [Explain the sta...]
-> Request 55 started [Write a sci-fi ...]
-> Request 56 started [Explain the sta...]
-> Request 57 started [Write a compreh...]
-> Request 58 started [Explain the sta...]
-> Request 59 started [Write a compreh...]
-> Request 60 started [Write a sci-fi ...]
-> Request 61 started [Explain the con...]
-> Request 62 started [Write a sci-fi ...]
-> Request 63 started [Write a sci-fi ...]
-> Request 64 started [Write a sci-fi ...]
-> Request 65 started [Explain the con...]
-> Request 66 started [Write a compreh...]
-> Request 67 started [Explain the con...]
-> Request 68 started [Write a compreh...]
-> Request 69 started [Explain the con...]
-> Request 70 started [Compare and con...]
-> Request 71 started [Write a compreh...]
```

```
 -> Request 72 started [Write a sci-fi ...]
 -> Request 73 started [Write a compreh...]
 -> Request 74 started [Explain the con...]
 -> Request 75 started [Write a sci-fi ...]
 -> Request 76 started [Compare and con...]
 -> Request 77 started [Write a sci-fi ...]
 -> Request 78 started [Explain the sta...]
 -> Request 79 started [Write a compreh...]
 -> Request 80 started [Explain the con...]
 -> Request 81 started [Explain the con...]
 -> Request 82 started [Explain the sta...]
 -> Request 83 started [Compare and con...]
 -> Request 84 started [Write a compreh...]
 -> Request 85 started [Write a sci-fi ...]
 -> Request 86 started [Write a sci-fi ...]
 -> Request 87 started [Explain the con...]
 -> Request 88 started [Write a sci-fi ...]
 -> Request 89 started [Explain the sta...]
 -> Request 90 started [Explain the sta...]
 -> Request 91 started [Write a sci-fi ...]
 -> Request 92 started [Explain the sta...]
 -> Request 93 started [Write a compreh...]
 -> Request 94 started [Explain the sta...]
 -> Request 95 started [Compare and con...]
 -> Request 96 started [Compare and con...]
 -> Request 97 started [Write a compreh...]
 -> Request 98 started [Compare and con...]
 -> Request 99 started [Write a compreh...]
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
.....................................................................
```

.....
.....
.....
.....
.....
.....
.....
.....
..... [Req 65 Done]
.....
..... [Req 87 Done]
.....
..... [Req 26 Done]
.....
..... [Req 18 Done]
..... [Req
69 Done]
..... [Req 48 Done]
..... [Req 32 Done]
..... [Req 5 Done]
..... [Req 15 Done]
.. [Req 39 Done]
..... [Req 67 Done]
..... [Req 11 Done]
.....
..... [Req 61 Done]
.....
..... [Req 36 Done]
..... [Req 80 Done]
 [Req 74 Done]
..... [Req 8 Done]
..... [Req 81 Done]
.....
.....
.....
.....
.....
..... [Req 0 Done]
.... [Req 1 Done]
. [Req 13 Done]
 [Req 9 Done]
 [Req 12 Done]
 [Req 4 Done]
 [Req 7 Done]
 [Req 6 Done]
 [Req 3 Done]
 [Req 2 Done]
 [Req 10 Done]

```
 [Req 19 Done]
 [Req 14 Done]
. [Req 17 Done]
 [Req 16 Done]
.. [Req 27 Done]
 [Req 23 Done]
 [Req 25 Done]
 [Req 22 Done]
 [Req 21 Done]
 [Req 28 Done]
 [Req 20 Done]
 [Req 24 Done]
 [Req 29 Done]
.. [Req 30 Done]
 [Req 34 Done]
 [Req 35 Done]
 [Req 37 Done]
 [Req 33 Done]
 [Req 31 Done]
. [Req 46 Done]
 [Req 44 Done]
 [Req 40 Done]
 [Req 42 Done]
 [Req 41 Done]
 [Req 38 Done]
 [Req 45 Done]
 [Req 43 Done]
.. [Req 54 Done]
 [Req 53 Done]
 [Req 52 Done]
 [Req 50 Done]
 [Req 55 Done]
 [Req 49 Done]
 [Req 47 Done]
 [Req 51 Done]
 [Req 64 Done]
 [Req 57 Done]
 [Req 60 Done]
 [Req 63 Done]
 [Req 62 Done]
 [Req 59 Done]
 [Req 56 Done]
 [Req 58 Done]
 [Req 71 Done]
.. [Req 72 Done]
 [Req 68 Done]
 [Req 73 Done]
 [Req 66 Done]
```

```
[Req 70 Done]
[Req 76 Done]
[Req 82 Done]
[Req 84 Done]
[Req 77 Done]
[Req 85 Done]
[Req 75 Done]
[Req 83 Done]
[Req 79 Done]
[Req 78 Done]
[Req 88 Done]
[Req 90 Done]
[Req 86 Done]
[Req 89 Done]
[Req 91 Done]
[Req 92 Done]
[Req 94 Done]
[Req 98 Done]
[Req 95 Done]
[Req 97 Done]
[Req 93 Done]
[Req 96 Done]
[Req 99 Done]


=== RANDOMIZED STRESS TEST RESULTS ===
Avg Latency (TTFT): 5.9710s
Avg Speed (TPS):    25.92 tokens/sec (Per user)
Total Throughput:   2287.55 tokens/sec (Aggregate)
Total Tokens:       178503
```

Implementation of the solution

Phase 1: System Preparation & Drivers

Objective: Ensure the physical hardware is ready and NVIDIA drivers are loaded.

1. **Update System:**

Shell

```
sudo apt-get update && sudo apt-get upgrade -y
```

2. **Install NVIDIA Drivers & CUDA:**

Shell

```
# Disable Noveau
cat <<EOF | sudo tee /etc/modprobe.d/blacklist-nouveau.conf
blacklist nouveau
options nouveau modeset=0
EOF

# Install NVIDIA Drivers (open) and Cuda
sudo apt-get install -y build-essential curl git jq
linux-headers-$(uname -r)
wget
https://developer.download.nvidia.com/compute/cuda/repos/ubunt
u2404/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt-get update
sudo apt-get install -y cuda-toolkit nvidia-open nvidia-gds

# Disable HMM for NVIDIA UVM
UVM_CONF_FILE="/etc/modprobe.d/nvidia-uvm.conf"
sudo bash -c "cat > $UVM_CONF_FILE" <<EOF
options nvidia_uvm uvm_disable_hmm=1
EOF
```

3. **Reboot:**

Shell

```
sudo reboot
```

4. **Configure MiG on the second GPU:**

```shell
# 1. Enable MIG on GPU 1 (ID: 1)
sudo nvidia-smi -i 1 -mig 1


# 2. Reset the GPU to apply the change

sudo nvidia-smi --gpu-reset -i 1


# 3. Create the Slices (Profile strings for 96GB Card)

# We need: 1x 48GB (4 compute slices) and 2x 24GB (2 compute slices each)

sudo nvidia-smi mig -cgi 5,14,14 -C -i 1

# 4. Verify the setup

nvidia-smi -L

# You should see GPU 0 with no MIG UUIDs, and GPU 1 with 3 MIG UUIDs.
```

Phase 2: OpenSearch Snap Setup (DB Layer)

Objective: Install the Vector Database on the bare metal OS via snap.

1. **Install the Snap:**

```shell

sudo snap install opensearch --channel=2/stable
```

```
# Grant permission to access system resources
sudo snap connect opensearch:process-control
```

2. **Tune Kernel Parameter for OpenSearch:**

```
Shell

echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

3. **Configure Network Binding:**

```
Shell

sudo snap set opensearch network-host="0.0.0.0"
```

4. **Generate Certificates:** We use the node name **`cm0`**. This name will be used for the SSL connection. These passwords are used as samples, please change them with stronger ones.

```
Shell

sudo snap run opensearch.setup \
  --node-name cm0 \
  --node-roles cluster_manager,data \
  --tls-priv-key-root-pass root1234 \
  --tls-priv-key-admin-pass admin1234 \
  --tls-priv-key-node-pass node1234 \
  --tls-init-setup yes
```

5. **Start the Database & Inizialize security:**

```
Shell

sudo snap start opensearch.daemon

# Initialize the security index (Default user: admin / pass: admin)
sudo snap run opensearch.security-init --tls-priv-key-admin-pass=admin1234
```

6. **Extract Root CA:** We need this file to allow our AI Apps to trust the database.

```
Shell

sudo cp /var/snap/opensearch/current/etc/opensearch/certificates/root-ca.pem ./root-ca.pem
sudo chown $USER:$USER root-ca.pem
```

Phase 3: Kubernetes & GPU Orchestration

Objective: Set up the container platform, GPU operator and enable GPU Time-Slicing so we can run multiple models on one card.

1. **Install Canonical Kubernetes:**

```
Shell

sudo snap install k8s --classic --channel=1.32-classic/stable
sudo k8s bootstrap
sudo k8s status --wait-ready
```

2. **Install Helm:**

```shell
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/master/scripts/get
-helm-3
chmod 700 get_helm.sh
./get_helm.sh
```

3. **Set KUBECONFIG:**

```shell
mkdir -p ~/.kube
sudo k8s config | sudo tee ~/.kube/config > /dev/null
sudo chmod 600 ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
export KUBECONFIG=~/.kube/config
```

1. **Deploy GPU Operator with MIG:**

```shell
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update

helm install gpu-operator nvidia/gpu-operator \
  -n gpu-operator \
  --create-namespace \
  --version=v25.3.4 \
  --set driver.enabled=false \
  --set mig.strategy=mixed \
  --set devicePlugin.config.default=any
```

Phase 4: Security & Networking

Objective: Create a bridge between Canonical Kubernetes and OpenSearch.

1. **Define Variables**

```shell
export HOST_IP=$(hostname -I | awk '{print $1}')
```

2. **Create Kubernetes Secrets:** if you changed the default admin password, please change it accordingly

```shell
# OpenSearch Trust (The CA we extracted earlier)
sudo k8s kubectl create secret generic opensearch-ca
--from-file=ca.crt=./root-ca.pem

# OpenSearch Login
sudo k8s kubectl create secret generic opensearch-creds \
  --from-literal=username="admin"
--from-literal=password="admin"
```

3. **Create Service Bridge (`cm0`):** This maps the DNS name `cm0` inside Kubernetes to your Host's Physical IP. This is performed for SSL verification.

```shell
cat <<EOF | sudo k8s kubectl apply -f -
apiVersion: v1
kind: Service
metadata: {name: cm0}
spec: {ports: [{protocol: TCP, port: 9200, targetPort: 9200}]}
---
apiVersion: v1
```

```
kind: Endpoints
metadata: {name: cm0}
subsets: [{addresses: [{ip: "$HOST_IP"}], ports: [{port:
9200}]}]
EOF
```

Phase 5: Application Deployment

Objective: Deploy the Chat Model, Embedding Model, and UI.

1. Deploy Chat Model (vLLM with GPT-OSS-120B - 4bit quantization)

We use `vLLM` running `openai/gpt-oss-120b`. This is a high-performance open model that does not require API tokens.

Shell

```
cat <<EOF | sudo k8s kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-generalist
  labels: {app: vllm-generalist}
spec:
  replicas: 1
  selector: {matchLabels: {app: vllm-generalist}}
  template:
    metadata: {labels: {app: vllm-generalist}}
    spec:
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
        resources:
          limits: {nvidia.com/gpu: 1} # Full 96GB GPU
        env:
```

```
        - {name: HUGGING_FACE_HUB_TOKEN, value:
"your_token_here"}
        args:
        - "--model"
        - "openai/gpt-oss-120b"
        - "--trust-remote-code"
        # --- FIX: Removed explicit quantization flag ---
        # vLLM will auto-detect 'mxfp4' from the config
        - "--gpu-memory-utilization"
        - "0.95"
        - "--max-model-len"
        - "32768"
        - "--served-model-name"
        - "gpt-oss-120b"
        - "--port"
        - "8000"
        ports: [{containerPort: 8000}]
        volumeMounts:
        - {name: model-cache, mountPath:
/root/.cache/huggingface}
      volumes:
      - name: model-cache
        hostPath: {path: /data/hf-cache, type:
DirectoryOrCreate}
---
apiVersion: v1
kind: Service
metadata: {name: vllm-generalist-service}
spec:
  type: ClusterIP
  selector: {app: vllm-generalist}
  ports: [{protocol: TCP, port: 80, targetPort: 8000}]
EOF
```

2. Deploy Chat Model Specialist (Coder)

Running `Qwen2.5-Coder-32B-Instruct-AWQ` on the 48 GiB MIG slice.

Shell

```
cat <<EOF | sudo k8s kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: vllm-coder}
spec:
  replicas: 1
  selector: {matchLabels: {app: vllm-coder}}
  template:
    metadata: {labels: {app: vllm-coder}}
    spec:
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
        resources:
          limits: {nvidia.com/mig-2g.48gb: 1} # 48GB Slice
        env:
        - {name: HUGGING_FACE_HUB_TOKEN, value:
"your_token_here"}
        # --- MIG STABILITY FLAGS ---
        - {name: NCCL_P2P_DISABLE, value: "1"}
        - {name: NCCL_IB_DISABLE, value: "1"}
        - {name: VLLM_WORKER_MULTIPROC_METHOD, value: "spawn"}
        args:
        - "--model"
        - "Qwen/Qwen2.5-Coder-32B-Instruct-AWQ"
        - "--quantization"
        - "awq"
        - "--gpu-memory-utilization"
        - "0.90"
        - "--max-model-len"
        - "16384"
        - "--enforce-eager"
        - "--served-model-name"
        - "coder-32b"
        ports: [{containerPort: 8000}]
---
apiVersion: v1
kind: Service
metadata: {name: vllm-coder-service}
```

```
spec:
  ports: [{port: 80, targetPort: 8000}]
  selector: {app: vllm-coder}
EOF
```

3. Deploy Small Model

Running Qwen2.5-7B-Instruct on a 24GiB MIG slice

Shell

```
cat <<EOF | sudo k8s kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-small
  labels: {app: vllm-small}
spec:
  replicas: 1
  selector: {matchLabels: {app: vllm-small}}
  template:
    metadata: {labels: {app: vllm-small}}
    spec:
      containers:
      - name: vllm
        image: vllm/vllm-openai:latest
        resources:
          limits: {nvidia.com/mig-1g.24gb: 1}
        args:
        - "--model"
        - "Qwen/Qwen2.5-7B-Instruct"  # <--- No Auth Required
        - "--max-model-len"
        - "32768"
        - "--gpu-memory-utilization"
        - "0.90"
        - "--port"
        - "8000"
        - "--trust-remote-code"
        ports: [{containerPort: 8000}]
        # Token env var is removed/ignored for ungated models
        volumeMounts:
        - {name: model-cache, mountPath: /root/.cache/huggingface}
      volumes:
      - name: model-cache
        hostPath: {path: /data/hf-cache, type: DirectoryOrCreate}
```

```
EOF
```

4. Deploy Embedding and Reranker

Running `BAAI/bge-m3` and `bge-reranker-v2-m3` on the CPU as the embedding models are not yet supported for blackwell. As soon as they will be supported, they will be deployed on the two remaining MIG slices of 24GiB

Shell

```
cat <<EOF | sudo k8s kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: tei-embedding}
spec:
  replicas: 1
  selector: {matchLabels: {app: tei-embedding}}
  template:
    metadata: {labels: {app: tei-embedding}}
    spec:
      containers:
      - name: infinity
        image: michaelf34/infinity:0.0.77
        resources:
          requests: {cpu: "24", memory: "16Gi"}
        env:
        - {name: HUGGING_FACE_HUB_TOKEN, value:
"your_token_here"}
        - {name: CUDA_VISIBLE_DEVICES, value: ""}
        - {name: OMP_NUM_THREADS, value: "24"}
        ports: [{containerPort: 80}]
        volumeMounts:
        - {name: model-cache, mountPath: /data}
        args:
        - "v2"
        - "--model-id"
        - "BAAI/bge-small-en-v1.5"
```

```
        - "--device"
        - "cpu"
        - "--port"
        - "80"
      volumes:
      - name: model-cache
        hostPath: {path: /data/hf-cache, type:
DirectoryOrCreate}
---
apiVersion: v1
kind: Service
metadata: {name: tei-embedding-service}
spec:
  ports: [{port: 80, targetPort: 80}]
  selector: {app: tei-embedding}

# --- Reranker Model (Fast/Base on CPU) ---
apiVersion: apps/v1
kind: Deployment
metadata: {name: tei-reranker}
spec:
  replicas: 1
  selector: {matchLabels: {app: tei-reranker}}
  template:
    metadata: {labels: {app: tei-reranker}}
    spec:
      containers:
      - name: infinity
        image: michaelf34/infinity:0.0.77
        resources:
          requests: {cpu: "12", memory: "8Gi"}
        env:
        - {name: HUGGING_FACE_HUB_TOKEN, value:
"your_token_here"}
        - {name: CUDA_VISIBLE_DEVICES, value: ""}
        - {name: OMP_NUM_THREADS, value: "12"}
        ports: [{containerPort: 80}]
        volumeMounts:
        - {name: model-cache, mountPath: /data}
```

```
        args:
        - "v2"
        - "--model-id"
        - "BAAI/bge-reranker-base"
        - "--device"
        - "cpu"
        - "--port"
        - "80"
      volumes:
      - name: model-cache
        hostPath: {path: /data/hf-cache, type:
DirectoryOrCreate}
---
apiVersion: v1
kind: Service
metadata: {name: tei-reranker-service}
spec:
  ports: [{port: 80, targetPort: 80}]
  selector: {app: tei-reranker}
EOF
```

5. Deploy Open WebUI (Fontend)

Shell

```
cat <<EOF | sudo k8s kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: open-webui-deployment}
spec:
  replicas: 1
  selector: {matchLabels: {app: open-webui}}
  template:
    metadata: {labels: {app: open-webui}}
    spec:
      volumes:
      - name: ca-volume
```

```
        secret: {secretName: opensearch-ca}
      containers:
      - name: open-webui
        image: ghcr.io/open-webui/open-webui:main
        ports: [{containerPort: 8080}]
        volumeMounts:
        - {name: ca-volume, mountPath:
/etc/ssl/certs/opensearch, readOnly: true}

        # --- Prevent bug
https://github.com/open-webui/open-webui/issues/20649 ---
        command: ["/bin/bash", "-c"]
        args:
          - |
            pip install "opensearch-py<3.0.0"
            bash /app/backend/start.sh

        env:
        # --- LLM Connections ---
        - name: OPENAI_API_BASE_URL
          value:
"http://vllm-generalist-service:80/v1;http://vllm-coder-servic
e:80/v1;http://vllm-small-service:80/v1"
        - name: OPENAI_API_KEY
          value: "EMPTY"

        # --- Embedding (UPDATED to Small) ---
        - {name: RAG_EMBEDDING_ENGINE, value: "openai"}
        - {name: RAG_OPENAI_API_BASE_URL, value:
"http://tei-embedding-service:80"}
        - {name: RAG_OPENAI_API_KEY, value: "dummy"}
        - {name: RAG_EMBEDDING_MODEL, value:
"BAAI/bge-small-en-v1.5"}

        # --- Reranker (UPDATED to Base) ---
        - {name: ENABLE_RAG_RERANKING, value: "True"}
        - {name: RAG_RERANKING_ENGINE, value: "openai"}
        - {name: RERANKING_OPENAI_API_BASE_URL, value:
"http://tei-reranker-service:80"}
```

```
        - {name: RERANKING_OPENAI_API_KEY, value: "dummy"}
        - {name: RAG_RERANKING_MODEL, value:
"BAAI/bge-reranker-base"}

        # --- RAG Config ---
        - {name: ENABLE_RAG_WEB_SEARCH, value: "True"}
        - {name: VECTOR_DB, value: "opensearch"}
        - {name: OPENSEARCH_URI, value: "https://cm0:9200"}
        - {name: OPENSEARCH_SSL_VERIFY, value: "client"}
        - {name: OPENSEARCH_CA_BUNDLE_PATH, value:
"/etc/ssl/certs/opensearch/ca.crt"}
        - {name: OPENSEARCH_USERNAME, valueFrom:
{secretKeyRef: {name: opensearch-creds, key: username}}}
        - {name: OPENSEARCH_PASSWORD, valueFrom:
{secretKeyRef: {name: opensearch-creds, key: password}}}
        - {name: CHUNK_SIZE, value: "2000"}
        - {name: CHUNK_OVERLAP, value: "200"}
        - {name: RAG_TOP_K, value: "10"}
        - {name: RAG_RERANKING_TOP_K, value: "5"}
EOF
```

Phase 6: Verification

You can perform the same steps shown for the NIM scenario